

MCS-023 (DBMS)

1. Explain the Two-Phase Locking Protocol. How does it guarantee serializability?

Ans. The **Two-Phase Locking (2PL) Protocol** is a widely used method in database systems to ensure *serializability*, which is the gold standard for correct transaction execution. It works by regulating how transactions acquire and release locks on data items.

Phases of Two-Phase Locking

There are **two key phases** in this protocol:

1. **Growing Phase:**
 - A transaction may obtain locks (shared for reading, exclusive for writing)
 - It **cannot release any lock** during this phase
 - This phase continues until the transaction acquires all the locks it needs
2. **Shrinking Phase:**
 - Once the transaction releases its first lock, it enters this phase
 - It **cannot acquire any new locks** from this point onward

Guaranteeing Serializability

2PL guarantees **conflict serializability**, which means the outcome of executing transactions concurrently will be the same as some serial execution (one transaction at a time) of those transactions. This happens because:

- Transactions are prevented from acquiring locks once they've started releasing them, avoiding cyclic dependencies
- This restriction avoids deadlock-prone interleavings that could violate serial order
- The resulting **schedule forms a serializable order**, usually represented by an acyclic precedence graph

There are variations like **Strict 2PL** (which also holds all exclusive locks until the transaction commits) that further help with recoverability and cascading aborts.

2. What is deadlock and how can it be prevented? Describe common deadlock-prevention schemes.

Ans. A **deadlock** in a database system (or any concurrent system) occurs when a group of transactions are each waiting for a resource that another transaction in the group holds, and none can proceed. It's like four cars stuck at a four-way stop, each waiting for the other to move—no one can go anywhere.

Conditions for Deadlock

Deadlock arises if these **four conditions** hold simultaneously:

1. **Mutual exclusion** – Resources cannot be shared; only one process can use a resource at a time.
2. **Hold and wait** – A process holding a resource may request additional ones.
3. **No preemption** – Resources can't be forcibly taken away.

4. **Circular wait** – A closed chain of processes exists where each process holds at least one resource the next process needs.

Deadlock Prevention Schemes

To **prevent deadlocks**, you need to ensure that at least one of the above conditions is never allowed to hold. Here are common strategies:

1. Resource Ordering

- Impose a global order on all resource types.
- Transactions must request resources in this order.
- Breaks the circular wait condition.

2. Wait-Die and Wound-Wait Schemes

These are **non-preemptive** strategies used with timestamps to control lock acquisition:

- **Wait-Die**: Older transactions wait; younger ones are aborted (“die”).
- **Wound-Wait**: Older transactions “wound” (preempt) younger ones, forcing them to roll back.

3. Timeouts

- Transactions wait for a limited time when requesting a lock.
- If the lock isn't acquired in time, the transaction is rolled back.

4. Prevention via Lock Protocols

- Use protocols that avoid hold-and-wait by requiring all locks be acquired upfront (can hurt concurrency though).

3. Compare primary, secondary and clustering indexes. Which of these indexes are dense and which are not? How is implementation of clustering indexes performed?

Ans. Let's break this down so it's crystal clear! Indexes in databases are like the index of a book—they help speed up data retrieval. But not all indexes are built the same. Here's how **primary**, **secondary**, and **clustering indexes** differ:

◆ Primary Index

- **Definition**: Built on a table's *primary key*—the main unique identifier.
 - **Structure**: One index entry per data block, not per row.
 - **Density**: **Sparse Index** – because each entry corresponds to a block, not every record.
 - **Ordering**: The data records are physically sorted according to this index.
-

◆ Secondary Index

- **Definition:** Built on *non-primary key* attributes, possibly allowing duplicates.
 - **Structure:** Contains a pointer to every matching record.
 - **Density:** **Dense Index** – since it includes an entry for every record.
 - **Ordering:** Data is **not** sorted by this attribute.
-

◆ Clustering Index

- **Definition:** Applied when data is *physically ordered* by a non-primary key column.
 - **Structure:** One entry for each distinct key value, pointing to a block of tuples (records) with that value.
 - **Density:** Usually a **Sparse Index**, unless each block contains exactly one record.
 - **Ordering:** Data records in the table are ordered to match the clustering attribute.
-

✂ Implementation of Clustering Index

Here's how it typically works:

1. **Sort the table** based on the clustering attribute.
2. **Group tuples** that share the same clustering key into contiguous blocks.
3. **Create the index** with one entry per key value pointing to the start of each block.

This organization makes **range queries blazing fast** for the clustering key but allows only **one clustering index per table**, since you can't physically order the table by multiple columns.

4. How do B-tree indexes differ from binary search tree indexes?

Ans. Great question! While both **B-trees** and **Binary Search Trees (BSTs)** are used for searching sorted data efficiently, B-trees are **specifically optimized for databases and disk storage**, making them much more practical in those environments.

Here's how they differ:

◆ 1. Structure & Balance

- **B-tree:** A balanced **multi-way tree**. Each node can have **multiple keys and children**, leading to fewer levels.
- **BST:** A binary tree where each node has **only two children (left and right)** and just **one key**.

✓ *B-trees remain balanced by design*, whereas BSTs can become unbalanced (degenerate into a linked list) if data is inserted in sorted order.

◆ 2. Fan-out and Height

- **B-tree**: High fan-out = fewer levels = **faster disk access**.
 - **BST**: More levels = **slower lookup** especially for large data sets.
- ✓ Since B-trees access **disk blocks**, their wide nodes reduce the number of reads needed.
-

◆ 3. Disk Optimization

- **B-tree**: Nodes sized to fit into a single disk block. Ideal for **minimizing I/O**.
 - **BST**: In-memory structure; not efficient for disk-based databases.
- ✓ B-trees are **designed for persistence**, not just RAM-based use.
-

◆ 4. Use in Indexing

- **B-tree**: Used in **database indexing** and file systems (like MySQL, PostgreSQL, NTFS).
 - **BST**: Great for algorithm learning and memory-resident structures, but **rarely used in production databases**.
-

So in a nutshell: **B-trees are to disk what BSTs are to memory**—they both aim for fast search, but B-trees win hands down when scalability, balance, and disk efficiency matter.

5. What is a B⁺-tree? Why is a B⁺-tree better structure than a B-Tree for implementation of an index sequential file?

Ans. A **B⁺-tree** is an advanced form of a B-tree that is widely used in databases and file systems, particularly for implementing **index sequential access**. While it's based on the same principles of balanced multi-level indexing, it has some key design enhancements that make it more efficient for certain operations.

◆ What is a B⁺-Tree?

- It's a **balanced tree** structure where **all data records are stored at the leaf level**.
 - **Internal (non-leaf) nodes** store only keys for navigation, not actual data.
 - **Leaf nodes are linked** together using **pointers**, forming a **linked list** to support fast sequential access.
-

◆ B-Tree vs. B⁺-Tree: Key Differences

Feature	B-Tree	B ⁺ -Tree
Data location	Stored in both internal and leaf nodes	Stored only in leaf nodes
Sequential access	Slower (due to tree traversal)	Faster (via linked leaf nodes)
Range queries	Less efficient	Highly efficient via leaf chaining
Tree height	Slightly smaller	Slightly taller due to data layout
Redundancy	Minimal	Keys may be duplicated in internal nodes

✂ Why B⁺-Tree is Better for Index Sequential File

- Efficient Range Queries:**
 - Because of the linked leaf nodes, scanning records in order is fast and straightforward.
- Better Disk Access Patterns:**
 - Nodes are block-sized and optimized for disk I/O, minimizing the number of reads.
- Predictable Access Time:**
 - All data is at the same level (leaves), ensuring consistent access times.
- Separation of Search and Storage:**
 - Internal nodes stay small and memory-resident for quick searching; data remains in leaves.

So in essence, **B⁺-trees are built for scalability, locality, and ordered access**, making them the ideal choice for implementing index sequential files in large databases and storage systems.

- Write SQL commands for each of the following:**
 - Creation of views;**
 - Creation of sequences;**
 - Outer join;**
 - To give access permission to any user.**

Ans. Sure! Here's how you'd write SQL statements for each of those operations:

(i) Creation of Views

```
CREATE VIEW employee_details AS
SELECT emp_id, emp_name, department
FROM employees
WHERE status = 'Active';
```

This creates a **virtual table** (view) that filters only active employees from the `employees` table.

(ii) Creation of Sequences

```
CREATE SEQUENCE emp_seq
START WITH 1000
INCREMENT BY 1
MAXVALUE 9999
CYCLE;
```

This creates a **sequence generator** starting from 1000, incrementing by 1. Useful for auto-generating unique IDs.

(iii) Outer Join (Example: Left Outer Join)

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

This fetches all employees along with their department names, including those who aren't assigned to a department (**nulls for unmatched rows**).

(iv) Granting Access Permission

```
GRANT SELECT, INSERT ON employees TO username;
```

This gives the user (replace `username` with actual user name) the ability to **read and insert** into the `employees` table.

7. **Why is data replication useful in Distributed Database Management System (DDBMS)? Briefly discuss the terms 'Complete Replication' and 'Selective Replication'.**

Ans. In a **Distributed Database Management System (DDBMS)**, **data replication** is like having multiple photocopies of the same report placed in different locations—you get faster access, better reliability, and added safety.

◆ Why is Data Replication Useful?

1. **Improved Availability:**
If one site fails, other sites still have a copy—no system downtime.
2. **Faster Query Performance:**
Users can access data from a nearby location, reducing network latency.

3. **Fault Tolerance & Reliability:**

Replicas act as backups, helping recover from hardware/software failures.

4. **Load Balancing:**

Read operations can be distributed across replicas, reducing the load on any single node.

◆ **Types of Data Replication**

1. *Complete Replication*

- Every site in the distributed system holds a **full copy of the entire database**.
- **Pros:** Excellent availability and fault tolerance.
- **Cons:** High storage cost; complex and heavy synchronization when updates occur.

2. *Selective Replication*

- Only **certain portions** of the database are replicated at selected sites.
 - **Pros:** More efficient in terms of space and update traffic.
 - **Cons:** Slightly reduced fault tolerance and data availability for unreplicated parts.
-

In essence, replication helps DDBMS achieve **high availability and robustness**, and choosing between complete and selective replication depends on the trade-off between **performance, cost, and consistency**.

8. **Explain the term data replication and data fragmentation with suitable example.**

Certainly! Both **data replication** and **data fragmentation** are key strategies in **Distributed Database Management Systems (DDBMS)** to enhance performance, availability, and reliability—but they work in very different ways. Let's break them down with examples:

☐ **Data Replication**

Definition:

Replication is the process of **storing copies** of the same data on **multiple sites** in a distributed system.

Purpose:

To improve **availability, fault tolerance, and read performance**.

Example:

Suppose a company has offices in Delhi, Mumbai, and Bangalore. The **customer data** is replicated at all three locations. So, if a user in Bangalore wants to access customer information, they can do it locally—without pinging the server in Delhi.

Benefit:

If the Delhi server goes down, users in Mumbai and Bangalore can still access the data.

Data Fragmentation

Definition:

Fragmentation breaks the database into **smaller pieces (fragments)**, which are distributed across various locations. The fragments can be accessed independently.

Types of Fragmentation:

1. **Horizontal Fragmentation:** Rows are divided based on criteria.
2. **Vertical Fragmentation:** Columns are divided into subsets.

Example:

Let's say you have a table `Employees(emp_id, emp_name, branch, salary)`.

- You could **horizontally fragment** it so that:
 - Delhi branch employees are stored at the Delhi server,
 - Mumbai branch employees are stored at the Mumbai server, and so on.
- Or you could **vertically fragment** it:
 - Personal info (`emp_id, emp_name`) on one server,
 - Work details (`branch, salary`) on another.

Benefit:

Reduces query processing time by storing only the **relevant data closer to its point of use**.

In Summary:

- **Replication** = many copies of the same data at many places
- **Fragmentation** = breaking up data and spreading the pieces

9. What are integrity constraints? Explain the various types of integrity constraints.

Ans. **Integrity constraints** are rules enforced on a database to ensure the **accuracy, consistency, and validity** of the data. They help maintain **data integrity** by restricting the kind of data that can be stored or modified in the database tables. Think of them as the “ground rules” of a database—ensuring no rogue entries sneak in or corrupt what's already there.

Types of Integrity Constraints

1. Domain Constraint

- Ensures that the values entered in a column belong to a defined **domain** (i.e. specific data type, format, or range).
- *Example:* A column `age` can only store integer values between 0 and 120.

2. Entity Integrity Constraint

- Ensures that every table has a **primary key**, and no part of it can be null.
- *Why?* Because primary keys uniquely identify each record—nulls would break that rule.

3. Referential Integrity Constraint

- Maintains consistency between **foreign keys** and the **referenced primary key** in another table.
- *Example:* If `orders.customer_id` is a foreign key referencing `customers.id`, you can't insert an order for a non-existent customer.

4. Unique Constraint

- Ensures that all values in a particular column (or set of columns) are **distinct** across the table.
- *Used when:* You need uniqueness, but the column isn't the primary key.

5. Not Null Constraint

- Prohibits null values in a column.
- *Why?* Useful when a column is **mandatory** for every record, like `email` in a user profile.

6. Check Constraint

- Enforces an arbitrary **condition or logical expression** on a column's value.
- *Example:* `salary > 0` to make sure negative salaries don't creep in.

These constraints together form a *safety net* that protects your data from inconsistency, accidental deletion, or logical errors.

10. Describe the following client-server architectures with the help of a diagram: (i) 2-tier (ii) 3-tier

Ans. Sure! Let's explore both **2-tier** and **3-tier client-server architectures**, which are commonly used in designing software systems—especially in databases and web applications.

□ (i) Two-Tier Architecture

This is a **client-server model** with two layers:

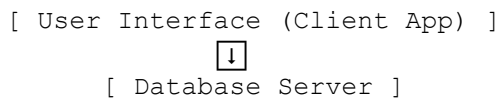
1. **Client Tier:** The front-end application (like a GUI) where users interact.
2. **Server Tier:** The back-end database that handles data storage and retrieval.

How it works:

The client communicates directly with the database server using SQL queries.

Example:

Bank employees using desktop software that directly connects to a central database.



Pros:

- Simple and fast for small systems.
- Easy to set up.

Cons:

- Poor scalability.
 - Hard to maintain as logic and UI are tightly coupled.
-

(ii) Three-Tier Architecture

This splits the application into **three distinct layers**:

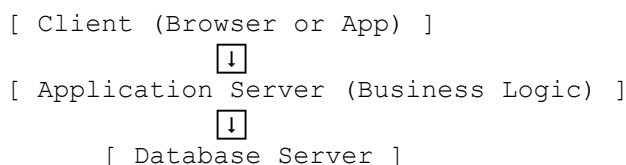
1. **Presentation Tier (Client)**: UI through which users interact.
2. **Logic Tier (Application Server)**: Business logic and rules.
3. **Data Tier (Database Server)**: Data management and storage.

How it works:

The client sends a request to the application server, which processes it and communicates with the database.

Example:

Online shopping websites like Amazon where your web browser (client) interacts with application logic (server-side processing), which in turn talks to a database.



Pros:

- More scalable and secure.
- Easier to manage and update.
- Ideal for web-based and enterprise applications.

Cons:

- Slightly more complex to develop.
-

11. What is a hashed file organization and inverted file organisation? What are its advantages and disadvantages?

Ans. Great question! Let's break down these two types of file organizations commonly used in databases and information retrieval systems:

Hashed File Organization

Definition

Data is stored at locations determined by a **hash function** applied to a key (e.g., a student ID or an employee number). This function maps the key to a specific storage location (bucket).

How it works:

- A key is input into a hash function
- The function returns an address
- The record is stored at that address

Advantages:

- ⚡ **Very fast access** to individual records using the key
- ✅ Efficient for **equality searches** (e.g., `WHERE ID = 12345`)
- 🔄 Simple to implement

Disadvantages:

- ❌ **Not suitable for range queries** (e.g., `ID BETWEEN 100 AND 200`)
 - 📦 Can cause **collisions** (two keys map to the same location), which need special handling
 - 🔄 **Rehashing** is needed if the file grows too big, which can be expensive
-

Inverted File Organization

Definition

Used mostly in text-based or document retrieval systems. It keeps an **inverted index**—a mapping from content (like keywords) to their corresponding records or documents.

How it works:

- Keywords or field values are extracted and stored in a separate index
- The index points to the record locations
- Often used in systems like search engines

Advantages:

- 🔍 Great for **complex queries** involving multiple fields or terms
- 🔍 Enables **fast full-text search** and Boolean queries (e.g., “apple AND orange”)

-  Useful in applications like **search engines** and **library systems**

Disadvantages:

-  Additional overhead to maintain the **inverted index**
-  Requires **extra storage**
-  Slightly **slower for inserting/updating records**, since the index must also be updated

12. **Explain the meaning of the following two keywords: Commit and Rollback. Write a code fragment for transferring money from account A to account B and show the uses of Commit and Rollback.**

Ans. Certainly! Let's break this down step by step.

COMMIT and ROLLBACK in Databases

These are **transaction control statements** in SQL that help maintain the integrity and consistency of data.

- **COMMIT**

 Saves all changes made during the transaction permanently in the database. After a COMMIT, the changes cannot be undone.

- **ROLLBACK**

 Undoes all the changes made in the current transaction, returning the database to the last consistent state before the transaction began.

These are crucial in **transactions**, where multiple operations need to succeed as one unit—or else not at all. Think of it as an “all or nothing” principle.

Money Transfer Code Example (Pseudocode in SQL)

Here's a code snippet that shows how these concepts are used when transferring ₹1000 from **Account A to Account B**:

```
BEGIN TRANSACTION;
```

```
-- Deduct from Account A
UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 'A';
```

```
-- Add to Account B
UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 'B';
```

```
-- Check if balances are valid (e.g., no negative balances)
IF (SELECT balance FROM accounts WHERE account_id = 'A') >= 0 THEN
    COMMIT; --  Make the changes permanent
```

```
ELSE
    ROLLBACK; -- ✗ Undo the whole transaction
END IF;
```

🧐 **Note:** In real-world applications, this logic would be implemented inside a database stored procedure or application code, with error handling for transaction failures, timeouts, etc.

13. What are Checkpoints? How does this technique of Checkpoints contribute to database recovery? Give suitable example.

Ans. Certainly! Let's demystify the concept of **Checkpoints** and why they're so important in database recovery. 🧐

❑ What Are Checkpoints?

A **checkpoint** is a **snapshot** of the current state of a database system, taken at regular intervals. It records all the essential information needed to recover the system in case of a crash or failure.

Think of it like saving your progress in a video game: if something goes wrong, you can return to the last saved point instead of starting all over again. 🎮

🔗 How Checkpoints Help in Database Recovery

When a database crashes:

- Instead of reading and processing the entire **log file** from the beginning (which could be massive), the system starts from the **last checkpoint**.
- Only the transactions **after** the checkpoint need to be examined for REDO (to apply completed transactions) or UNDO (to rollback incomplete ones).

This drastically **reduces recovery time**, improves **efficiency**, and helps maintain **data consistency**.

🔗 Example

Let's say we have a database performing the following transactions:

```
T1: Start → Update A → Commit
T2: Start → Update B
T3: Start → Update C → Commit
```

A **checkpoint** is created right after T1 commits.

❑ Now, imagine the system crashes before T2 and T3 complete.

During Recovery:

- The system starts scanning the logs from the **last checkpoint** (after T₁)
- T₂ is **undone** (it wasn't committed)
- T₃ is **redone** (it was committed)
- No need to look at T₁ because its effects are already safe

✓ Summary of Benefits

- ▶ Faster recovery after failure
- 📉 Less disk I/O during recovery
- □ Improved system reliability
- 🗃️ Efficient transaction log management

14. What is a data dictionary? What should be included in data dictionary?

Ans. 📝 Great question! Let's break it down.

□ What is a Data Dictionary?

A **data dictionary** is a centralized **repository of metadata**—it describes the data, not the data itself. Think of it like a *guidebook* that helps users, developers, and systems understand the structure, relationships, and attributes of data stored in a database.

It acts as a **blueprint** of your database, offering a clear map of what each piece of data means, how it's used, and where it's located.

📋 What Should Be Included in a Data Dictionary?

Here's a list of key elements typically found:

Component	Description
Table Name	Name of each table in the database
Column Name	Each field or attribute in a table
Data Type	Type of data allowed (e.g., INT, VARCHAR, DATE)
Length/Size	Maximum size of the data field
Default Values	Values used if none are provided during data entry
Constraints	Rules like PRIMARY KEY, FOREIGN KEY, NOT NULL, etc.
Indexes	Info on indexed fields for faster searching
Relationships	Links between tables (e.g., one-to-many, many-to-many)

Component	Description
Access Permissions	Who can read, write, or modify the data
Descriptions / Comments	Human-readable explanation of each table or field's purpose

Example

Let's say you have a table called `Students`:

Column Name	Data Type	Size	Description
student_id	INT	4	Unique student number
name	VARCHAR	50	Full name
dob	DATE	—	Date of birth
grade	CHAR	2	Grade or score

This table's structure would be stored in the data dictionary.

Why It Matters

-  Helps with **data consistency** and accuracy
-  Essential for **database designers and developers**
-  Facilitates **documentation, auditing, and troubleshooting**

15. **What is Optimization in DBMS? How does relational algebra help in Optimizing the Query? Discuss with suitable example. List the operators used in relational algebra and briefly explain them.**

Ans. Absolutely! Let's explore query optimization in DBMS and how relational algebra plays a crucial role in it. ✨

What is Optimization in DBMS?

Query Optimization is the process of selecting the most efficient way to execute a database query by considering different query execution plans. The goal is to reduce:

-  Time (response time or throughput)
-  Resource usage (CPU, memory, disk I/O)

The **query optimizer** analyzes multiple ways a query could be executed and picks the one with the lowest cost.

Role of Relational Algebra in Optimization

Relational Algebra provides a **theoretical foundation** for query processing. SQL queries are translated into **relational algebra expressions**, and these expressions can be **restructured or simplified** for better performance.

Example

Suppose we write an SQL query:

```
SELECT name
FROM Student
WHERE dept = 'CS' AND age > 18;
```

In relational algebra, it looks like:

```
 $\pi_{name} (\sigma_{dept='CS' \wedge age>18} (Student))$ 
```

We can optimize this by **pushing selections down**:

```
 $\pi_{name} (\sigma_{age>18} (\sigma_{dept='CS'} (Student)))$ 
```

This reduces the number of records handled at each step—making it faster.

Operators in Relational Algebra

Here’s a quick reference of key relational algebra operators:

Operator	Symbol	Description
Selection	σ	Filters rows based on condition ($\sigma_{age>18}$)
Projection	π	Selects specific columns ($\pi_{name, age}$)
Union	$\cup$	Combines tuples from two relations (removes duplicates)
Set Difference	$-$	Tuples in one relation but not in the other
Cartesian Product	$\times$	Combines every tuple of one relation with every tuple of another
Rename	ρ	Renames the relation or its attributes
Intersection	$\cap$	Common tuples in two relations
Join	$\bowtie$	Combines related tuples from two relations
Division	$\div$	Useful for “for all” queries (e.g. who took <i>all</i> courses)

Summary

Relational algebra helps by:

- Providing a **logical plan** that the optimizer can refine

- Allowing **algebraic transformations** (like pushing selections down, using joins effectively)
- Making complex queries **simpler and faster**

16. Explain log-based recovery scheme and shadow paging recovery scheme with the help of an example.

Ans. Certainly! Let's explore two fundamental recovery techniques used in database management systems (DBMS): **Log-Based Recovery** and **Shadow Paging**. These are essential for ensuring **data consistency and durability**, especially in the face of system crashes or failures. 📄 ↺

📄 1. Log-Based Recovery Scheme

This approach uses a **transaction log** to record all changes made to the database. The log ensures that committed transactions can be **redone** and uncommitted ones can be **undone** during recovery.

🔧 How It Works:

- Every operation (e.g., update, insert) is recorded in a log **before** it's applied to the database. This is called **Write-Ahead Logging (WAL)**.
- The log entries typically look like:
 - `<T1, START>`
 - `<T1, A, 500, 450>` -- (Transaction T1 updates A from 500 to 450)
 - `<T1, COMMIT>`
- In case of crash:
 - **Redo** all transactions that have COMMIT in the log
 - **Undo** all transactions that had START but no COMMIT

💡 Example:

Imagine this transaction:

```
-- T1 wants to transfer ₹50 from A to B
START TRANSACTION
UPDATE A SET balance = balance - 50
UPDATE B SET balance = balance + 50
COMMIT
```

Corresponding log entries:

```
<T1, START>
<T1, A, 1000, 950>
<T1, B, 500, 550>
<T1, COMMIT>
```

If a crash happens *after* the COMMIT, we **redo** T1 during recovery to ensure both balances are correct.

📄 2. Shadow Paging

This technique avoids logs entirely by using **page-level duplication**. It maintains a **shadow copy** of the database that remains unchanged until a transaction is committed.

How It Works:

- The database has two page tables:
 - **Current Page Table (CPT)**: Points to the actual working copy
 - **Shadow Page Table (SPT)**: Points to the last consistent state
- All updates are made to pages pointed to by CPT
- If a transaction **commits**, CPT becomes the new SPT
- If a transaction **fails**, CPT is discarded and the system resumes using SPT

Example:

Suppose Page 3 contains Account A's balance.

1. Before update:
 - SPT → Page 3 (Balance: ₹1000)
2. Transaction modifies balance to ₹950:
 - CPT → *New Page 7* (modified data)
3. On commit:
 - CPT becomes new SPT
 - Old page can be recycled

If system crashes **before** the commit, CPT is discarded, and the database reverts to the unmodified shadow copy.

Comparison

Feature	Log-Based Recovery	Shadow Paging
Uses Transaction Log	☑ Yes	✗ No
Performance Overhead	Medium (log I/O)	Low for reads, High for updates
Crash Recovery	Redo/Undo using log	Revert to shadow pages
Complexity	Moderate	Higher implementation cost
Commit Handling	Logs written & flushed	Page table switched
